

Giulio

Polymath Jr

07/25/2025

1st key fact: If you find a map $T(x)$ s.t. $\rho \xrightarrow{T} \mu$

($T\#\rho = \mu$ or $\rho(x) = J^T(x) \mu(T(x))$) $\Omega \subset \mathbb{R}^d$

and it happens that $T(x) = \nabla_x \varphi(x)$ where $\varphi(x): \Omega \rightarrow \mathbb{R}$ is convex

Brenier's Theorem

Let ρ and μ be two probability measures on $\mathbb{R}^d$. Assume:

- ρ is absolutely continuous with respect to Lebesgue measure (i.e., it has a density $\rho(x)$),
- $\Omega \subset \mathbb{R}^d$ is convex,
- The cost function is the squared Euclidean distance:

$$c(x, y) = \frac{1}{2} \|x - y\|^2$$

Then there exists a unique optimal transport map T pushing ρ to μ which minimizes the transport cost, and this optimal map is of the form:

$$T(x) = \nabla \varphi(x)$$

for some convex scalar function φ .

Geometric Intuition

If you think of ρ as mass (such as sand) on Ω , and you're trying to move it to match a target shape μ (such as a hole), the most efficient way to do it under squared distance cost is to push it along the gradient of a convex potential.

Mathematical Consequences

When $T = \nabla \varphi$, the Jacobian determinant change-of-variable formula becomes:

$$\rho(x) = \det(D^2 \varphi(x)) \cdot \mu(\nabla \varphi(x))$$

This relates the second derivative (Hessian) of the convex potential φ to the densities ρ and μ , and is used in Monge–Ampère equations in optimal transport.

Why Must $\Omega \subset \mathbb{R}^d$ Be Convex?

The domain $\Omega \subset \mathbb{R}^d$ being convex means for any two points $x_1, x_2 \in \Omega$, the line segment connecting them is also entirely contained in Ω . In symbols:

$$\forall x_1, x_2 \in \Omega, \forall \lambda \in [0, 1], \quad \lambda x_1 + (1 - \lambda)x_2 \in \Omega$$

To clarify, the expression

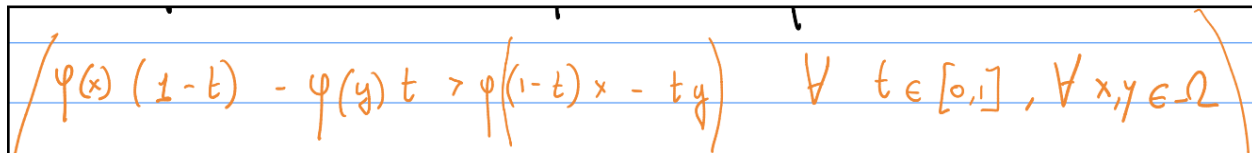
$$x(\lambda) = \lambda x_1 + (1 - \lambda)x_2$$

produces a point between x_1 and x_2 :

- If $\lambda = 0$, you're at x_2
- If $\lambda = 1$, you're at x_1
- If $\lambda = 0.5$, you're at the midpoint
- If $\lambda = 0.25$, you're 25% of the way from x_1 to x_2
- If $\lambda = 0.75$, you're 75% of the way from x_1 to x_2

This is why $\lambda x_1 + (1 - \lambda)x_2$ is called a **convex combination**, and as $\lambda \in [0, 1]$, it traces the full **closed line segment** between the two points.

Many results in convex analysis (for example the existence and uniqueness of solutions to Monge–Ampère equations) require convexity of the domain.



The image shows a handwritten equation in orange ink on lined paper. The equation is: $\varphi(x)(1-t) - \varphi(y)t > \varphi((1-t)x - ty) \quad \forall t \in [0,1], \forall x, y \in \Omega$. The equation is enclosed in large parentheses.

Understanding Convex Combinations and Convexity

Disclaimer: The equation written in the notes, namely

$$\varphi(x)(1-t) - \varphi(y)t > \varphi((1-t)x - ty),$$

is **incorrect**. It does not reflect the definition of convexity. The correct form involves convex combinations and is stated below in step 2.

Step 1: Convex Combination Definition

The expression

$$x(\lambda) = \lambda x_1 + (1 - \lambda)x_2$$

parametrizes the **line segment** between x_2 and x_1 . This is a **function that outputs a point** between two fixed vectors.

Note: this version of the convex combination puts x_1 first. That's totally valid — both:

$$\lambda x_1 + (1 - \lambda)x_2 \quad \text{and} \quad (1 - t)x + ty$$

are convex combinations. They're just written with different symbols and ordering.

Step 2: Use a General Convex Combination for Convexity

To define convexity of a function φ , we use:

$$\varphi((1-t)x + ty) \leq (1-t)\varphi(x) + t\varphi(y) \quad \forall t \in [0, 1], \forall x, y \in \Omega.$$

In other words the function evaluated at any point on the **line segment between x and y** is less than or equal to the straight-line interpolation between $\varphi(x)$ and $\varphi(y)$.

Handwritten note in a box: OR $\varphi(y) > \varphi(x) + \nabla \varphi(x)(y-x) \quad \forall x, y \in \Omega$

This equation says the function φ lies above its tangent hyperplane at every point x . **Note:** The expression

$$\varphi(y) \geq \varphi(x) + \nabla \varphi(x)(y-x)$$

does imply a dot product between the gradient vector $\nabla \varphi(x)$ and the displacement vector $y-x$. More formally, it should be written as:

$$\varphi(y) \geq \varphi(x) + \nabla \varphi(x)^\top (y-x) \quad \text{or} \quad \varphi(y) \geq \varphi(x) + \langle \nabla \varphi(x), y-x \rangle$$

Tangent Plane Formula

The equation of a tangent plane to a surface given by $z = f(x, y)$ at a point $(a, b, f(a, b))$ is:

$$z = f(a, b) + f_x(a, b)(x-a) + f_y(a, b)(y-b)$$

where f_x and f_y are the partial derivatives of f with respect to x and y , respectively.

Handwritten note in a box: OR H^φ is POS DEF $y^\top H^\varphi y > 0 \quad \forall y \in \mathbb{R}^d \quad H_{ij}^\varphi = \frac{\partial^2 \varphi}{\partial x_i \partial x_j} \quad \|x - T(x)\|^2$

$$S \succeq 0 \iff x^\top S x \geq 0 \quad \forall x \in \mathbb{R}^n \iff \lambda_i \geq 0 \quad \text{for all eigenvalues } \lambda_i \text{ of } S$$

Handwritten note in a box: then is OPTIMAL in the sense that $\nabla \varphi = \argmin_T \int_{\Omega} C(x, T(x)) p(x) dx$

Intuition Behind the Optimal Transport Cost

The map $T = \nabla\varphi$ is optimal in the sense that it minimizes the total transport cost:

$$\min_T \int_{\Omega} c(x, T(x)) \rho(x) dx,$$

where $\rho(x)$ is the probability density function on the domain $\Omega \subset \mathbb{R}^d$, and $c(x, T(x))$ is the cost of transporting mass from x to $T(x)$.

How to interpret $\rho(x)$:

- A single point x has zero measure, so the value $\rho(x)$ itself does *not* represent probability or mass.
- Instead, to estimate the probability of mass near x , we consider a small neighborhood of width δx and write:

$$\mathbb{P}(x \leq X \leq x + \delta x) \approx \rho(x)$$

Putting this into the transport cost integral:

- The expression $c(x, T(x)) \rho(x) dx$ gives the cost of moving the infinitesimal amount of mass $\rho(x) dx$ from x to $T(x)$.
- Integrating over the entire domain Ω sums up the cost of moving *all mass*, weighted according to the density at each location.

In the case of quadratic cost:

$$c(x, T(x)) = \|x - T(x)\|^2,$$

the total cost becomes:

$$\int_{\Omega} \|x - T(x)\|^2 \rho(x) dx.$$

This measures how far the mass is being moved, squared and weighted by how much mass is located at each $x \in \Omega$.

2nd key fact

Let $T_{\beta}^{(x)}$ be a parametric function

then it makes sense to, compute

$$\beta^* = \arg \min_{\beta} KL(J^{T_{\beta}^{(x)}} \mu(T_{\beta}^{(x)}) \mid \rho(x))$$

$KL(\nu_1, \nu_2) = \int \nu_1(x) \log\left(\frac{\nu_1(x)}{\nu_2(x)}\right) dx$

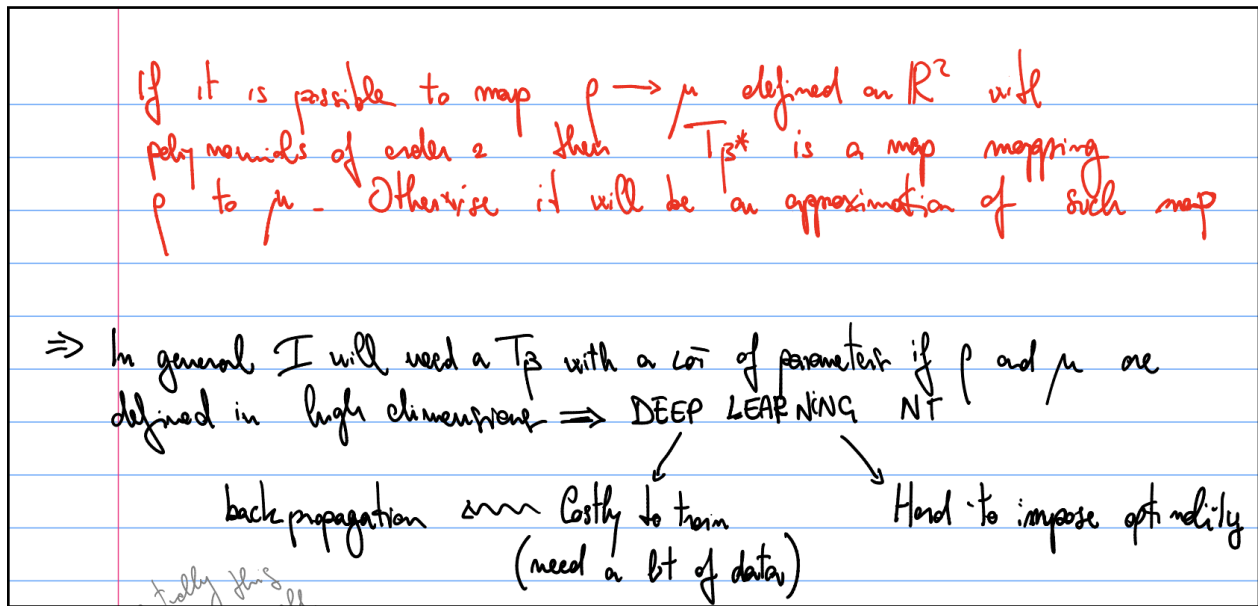
Eg. $T_{\beta} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$

in 2D

$$T_{\beta}(x, y) = \begin{bmatrix} \beta_1 + \beta_2 x + \beta_3 y + \beta_4 x^2 + \beta_5 y^2 + \\ \beta_6 xy \\ \beta_7 + \beta_8 x + \dots + \beta_{12} xy \end{bmatrix}$$

Given (usually $\mathcal{N}(0, I)$)

you are only given samples $\{x_i\}_{i=1, \dots, N}$



Why Use Neural Networks to Represent the Transport Map T^{β}

When you're working with **optimal transport** between two probability distributions ρ and μ , the goal is to learn a map

$$T^{\beta} : \mathbb{R}^d \rightarrow \mathbb{R}^d$$

that pushes ρ forward to μ , while ideally minimizing a transport cost (e.g., squared Euclidean distance). Now if ρ and μ are defined in **high-dimensional space** (e.g., $d = 100$ or more), then:

- T^{β} is a function from high-dimensional input to high-dimensional output.
- In general, such functions are **complex** and can exhibit intricate structures (e.g., nonlinearities, interactions between coordinates, etc.).
- Traditional function approximators (e.g., linear models, polynomials) often **fail** in these settings because they are too rigid.

Why Neural Networks (Deep Learning)?

Neural networks are **universal function approximators**—they can approximate a wide variety of functions using relatively compact representations, especially when:

- The data lives in high dimensions.
- The function to learn (here, the transport map T^{β}) has nontrivial structure.

By stacking multiple nonlinear layers, **deep nets** can:

- Capture complex relationships across many variables.
- Reuse patterns across dimensions.
- Efficiently model functions in high-dimensional spaces **without explicitly enumerating all variables**.

Thus, when you're trying to learn a map T^{β} between high-dimensional distributions, deep learning gives you a **flexible and powerful framework** to represent T^{β} , which you can train using data sampled from ρ and μ .

Neural Networks: Setup and Benefits

- You define $T^\beta(x) = \text{NN}_\beta(x)$.
- You choose the **architecture** (layers, activations, etc.), and let β represent all the trainable weights.
- Neural networks are **expressive enough** to approximate many kinds of functions.

Downsides of Deep Learning

Despite its flexibility, deep learning comes with some notable downsides:

- You must use **backpropagation** to train the model.
- It can be **costly to train**, especially in high dimensions, since you typically need a **large amount of data**.
- It is **hard to impose optimality constraints**—such as convexity or theoretical structure from optimal transport—on the learned neural network function.

Training neural networks: Setup

- Training data

$$S_n = \{(\bar{x}^{(i)}, y^{(i)})\}_{i=1}^n$$
$$\bar{x} \in \mathbb{R}^d \quad y \in \{-1, +1\}$$

- Neural network

$$\bar{\theta} = [w_{10}^{(1)}, w_{11}^{(1)}, \dots, w_{JK}^{(L)}]$$

$$h(\bar{x}; \bar{\theta}) = z^{(L+1)} = z_1^{(L+1)}$$

h represents your whole network

- Loss function

$$J(\bar{\theta}) = \frac{1}{n} \sum_{i=1}^n \text{Loss}(y^{(i)}, h(\bar{x}^{(i)}; \bar{\theta}))$$

eventually this could be a small ICNN (need a bit of data)

3rd key fact

Main difference with paper Tabak-Torrer.

Using the idea of the flow it is easier to impose optimality

$\{x_i^0\} \sim \rho(x) \quad x_i^0 \in \mathbb{R}^d$

$x_i^{n+1} = x_i^n + \beta \nabla F(z_i^{n+1})$ where $z_i^{n+1} = \|x_i^0 - \bar{x}^{n+1}\|$ and $f(z) = \frac{1}{z} \frac{dF(z)}{dz}$

RANDOM POINT (around which the elementary map is localized)

Each elementary map is a radially symmetric function (but it doesn't have to) AND is the gradient of a convex function (the elementary potential) F

Update Rule and Notation

We define an incremental transport map by updating each point $x_i \in \mathbb{R}^d$ via a sequence of elementary maps. The update rule at step n is:

$$x_i^{n+1} = x_i^n + \beta f(z_i^{n+1}) (x_i^0 - \bar{x}^{n+1})$$

Main idea

Find $T = T_n \circ \dots \circ T_1$ as a composition of elementary maps, easier to parametrize $T_k = T_{\beta^k}$

Explanation of Terms

- x_i^0 : the original sample from the source distribution ρ
- x_i^n : the position of point i after n transformation steps
- $\bar{x}^{n+1}$: a randomly sampled center point (from ρ) used to localize the next elementary map
- $z_i^{n+1} = \|x_i^0 - \bar{x}^{n+1}\|$: the Euclidean distance from the original point x_i^0 to the center
- $f(z)$: a scalar radial function controlling the magnitude of movement, defined below
- β : a scalar step size (learned or fixed)



Sampling Center Points $\bar{x}^{n+1}$

At each step $n+1$, you randomly pick a new center point $\bar{x}^{n+1} \sim \rho$.

• If, by chance, the same **original point** is selected as $\bar{x}^{n+1}$ again at some later step $m+1$, it **might have moved** since the first time it was selected.

• That is:

If $\bar{x}^{n+1} = x_j^0$ and later $\bar{x}^{m+1} = x_j^0$ again,

then the **value of that point has changed** because it has undergone several transformations:

$$x_j^0 \rightarrow x_j^1 \rightarrow x_j^2 \rightarrow \dots \rightarrow x_j^m$$

Radial Function Definition

We consider scalar potentials $F(x)$ of the form:

$$F(x) = \Phi(\|x - \bar{x}\|),$$

where:

- $\bar{x} \in \mathbb{R}^d$ is a fixed center point (e.g., sampled from the source distribution ρ),
- $r = \|x - \bar{x}\|$ is the radial distance from x to $\bar{x}$,
- $\Phi : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$ is a smooth convex scalar function.
- $F(x)$ is a radially symmetric and convex function.

Gradient of the Potential

To compute $\nabla F(x)$, we apply the chain rule:

$$\begin{aligned}
\nabla F(x) &= \frac{d}{dr} \Phi(r) \cdot \nabla r \\
&= \Phi'(r) \cdot \frac{x - \bar{x}}{\|x - \bar{x}\|} \\
&= \Phi'(r) \cdot \frac{x - \bar{x}}{r}.
\end{aligned}$$

We define the radial function:

$$f(r) := \frac{1}{r} \frac{d\Phi(r)}{dr},$$

so that the gradient becomes:

$$\nabla F(x) = f(r)(x - \bar{x}).$$

Step-by-step breakdown

Write the norm as a square root of an inner product:

$$r(x) = \|x - \bar{x}\| = \sqrt{(x - \bar{x})^\top (x - \bar{x})}$$

Differentiate using the chain rule:

Note: Since $r(x)$ is a scalar-valued function of a single vector variable x , computing the gradient of r is equivalent to differentiating with respect to x .

$$\frac{d}{dx} \sqrt{(x - \bar{x})^\top (x - \bar{x})}$$

Let's set $u(x) = (x - \bar{x})^\top (x - \bar{x})$. Then $r(x) = \sqrt{u(x)}$, and we use the chain rule:

$$\frac{d}{dx} r(x) = \frac{1}{2\sqrt{u(x)}} \cdot \frac{d}{dx} u(x)$$

Differentiate the inner function:

$$\frac{d}{dx} ((x - \bar{x})^\top (x - \bar{x})) = 2(x - \bar{x})$$

Put it all together:

$$\frac{d}{dx} \|x - \bar{x}\| = \frac{1}{2\|x - \bar{x}\|} \cdot 2(x - \bar{x}) = \frac{x - \bar{x}}{\|x - \bar{x}\|}$$

Clarifying the Derivative Step

Let's clarify where the 2 in the denominator came from in this step:

We are differentiating:

$$r(x) = \|x - \bar{x}\| = \sqrt{(x - \bar{x})^\top (x - \bar{x})}$$

Let's denote:

$$u(x) = (x - \bar{x})^\top (x - \bar{x}) \quad \text{so} \quad r(x) = \sqrt{u(x)}$$

$$\frac{d}{dx} \sqrt{u(x)} = \frac{1}{2\sqrt{u(x)}} \cdot \frac{d}{dx} u(x)$$

That 2 in the denominator comes from the derivative of the square root:

$$\frac{d}{dx} \sqrt{u} = \frac{1}{2\sqrt{u}} \cdot \frac{du}{dx}$$

Interpretation

This shows that the flow field defined by $f(r)(x - \bar{x})$ is the gradient of the convex function $F(x) = \Phi(\|x - \bar{x}\|)$. Since Φ is convex and composition with the norm preserves convexity, F is convex as well. Therefore, each such flow is guaranteed to be the gradient of a convex function.

Connection to Brenier's Theorem

Recall from Brenier's Theorem: Let ρ and μ be two probability measures on $\mathbb{R}^d$, and assume:

- ρ is absolutely continuous with respect to Lebesgue measure (i.e., has a density $\rho(x)$),
- the domain $\Omega \subset \mathbb{R}^d$ is convex,
- and the cost function is the squared Euclidean distance:

$$c(x, y) = \frac{1}{2} \|x - y\|^2,$$

Then there exists a unique optimal transport map T pushing ρ to μ which minimizes the transport cost, and this optimal map is of the form:

$$T(x) = \nabla \varphi(x)$$

for some convex function φ .

Therefore, the elementary maps used in our construction — which are gradients of convex scalar potentials — are consistent with the structural form of optimal transport maps prescribed by Brenier's Theorem.

Interpretation

Each elementary map is defined by a localized radial transformation centered at $\bar{x}^{n+1}$. The idea is to push the point x_i^n slightly in the direction from $\bar{x}^{n+1}$ to x_i^0 , scaled by a function of how far x_i^0 is from the center.

- The vector $x_i^0 - \bar{x}^{n+1}$ gives the direction of flow.
- The scalar function $f(z_i^{n+1})$ gives the magnitude of the flow.
- This flow is applied additively to the current position x_i^n , yielding x_i^{n+1} .

Note: The formulas

$$T(x) = \nabla\varphi(x) \quad \text{and} \quad x_i^{m+1} = x_i^m + \beta \nabla F(x_i^m)$$

represent the same underlying idea: both describe a transport map defined by the gradient of a convex function. The first is a continuous-time formulation from optimal transport, while the second is a discrete-time update used in iterative methods. They are two versions of the same gradient flow under different time settings.

One idea on OT

We need to impose the convexity of the potential $\rightarrow$ we want the map to be the gradient of a convex function. If we build the flow

$$z^{k+1} = z^k + \beta^k \nabla_x F(z^k) = z^0 + \sum_{i=1}^k \nabla_x F(z^i)$$

with $z^0 = x$ the starting position, $\beta \in R$, and F convex, then the convexity of the potential depends on the sign of β .

- When β^i is positive there is no problem (sum of convex functions is still convex)
- If β is negative we need to pay attention
- We only have the value $\sum_{i=1}^k \nabla_x F(z^i)$ at the sample points z^i
- A condition that $\sum_{i=1}^k \nabla_x F(z^i)$ should satisfy is that, at least, should interpolate a convex function

Recall that the gradient is a linear operator, so the gradient of a sum is equal to the sum of the gradients.

E.g. take $F(z) = z \operatorname{erf}\left(\frac{z}{\alpha}\right) + \frac{\alpha}{\sqrt{\pi}} e^{-\frac{z^2}{\alpha^2}}$

$\operatorname{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-x^2} dx$

$f(z) = \frac{\operatorname{erf}\left(\frac{z}{\alpha}\right)}{z}$

$\alpha = \text{bandwidth}$

How to compute it?

local in space

Q How do we compute β ? (Tabak-Turner)

$KL(p, p^{m+1}) = \int p(x) \log\left(\frac{p}{p^{m+1}}\right) dx$

$p(x) = \mu(T(x)) J(T(x))$

$\int p \log p - \int p \log p^{m+1}$

Monte Carlo Approximation

Decreasing $KL(p, p^{m+1}) \Leftrightarrow$ Increasing $\int p \log p^{m+1} dx \approx \frac{1}{N} \sum_{i=1}^N \log p^{m+1}(x_i)$

$= \frac{1}{N} \sum_{i=1}^N \log \mu(x_i^{m+1}) J(x_i^{m+1}) = \mathcal{L}(\beta)$

$T^{m+1}(x) = x^{m+1} = x^m + \beta f^{m+1}(x^m - \tilde{x}^{m+1})$

Choose β^{m+1} proportional to $\frac{\partial \mathcal{L}}{\partial \beta}$ (E.g. $\beta^{m+1} = \frac{\partial \mathcal{L}}{\partial \beta}$)

Computing the Step Size β^{m+1} in Particle Flow (Tabak-Turner)

We aim to compute the step size β^{m+1} that moves a particle cloud closer to a target distribution ρ using the iterative flow-based method proposed by Tabak and Turner. This is done by minimizing the Kullback-Leibler (KL) divergence between the target ρ and the current particle approximation ρ^{m+1} .

KL Divergence Objective

We aim to minimize:

$$\text{KL}(\rho, \rho^{m+1}) = \int \rho(x) \log \left(\frac{\rho(x)}{\rho^{m+1}(x)} \right) dx$$

which quantifies the discrepancy between the target density ρ and the current approximation ρ^{m+1} .

Expressing ρ^{m+1} via Pushforward

Let μ be a known base distribution, and let T^{m+1} be the transport map at iteration $m+1$. Then:

$$\rho^{m+1}(x) = \mu(T^{m+1}(x)) \cdot |J^{m+1}(x)|$$

where $J^{m+1}(x)$ denotes the Jacobian determinant of the transformation T^{m+1} .

Simplifying the KL Expression

Decomposing the KL divergence:

$$\text{KL}(\rho, \rho^{m+1}) = \int \rho(x) \log \rho(x) dx - \int \rho(x) \log \rho^{m+1}(x) dx$$

Since the first term is constant with respect to ρ^{m+1} , minimizing KL is equivalent to maximizing:

$$\int \rho(x) \log \rho^{m+1}(x) dx$$

Monte Carlo Approximation

We estimate this expectation using Monte Carlo samples $x_i^0 \sim \rho$, yielding:

$$\int \rho(x) \log \rho^{m+1}(x) dx \approx \frac{1}{N} \sum_{i=1}^N \log \rho^{m+1}(x_i^0)$$

Substituting the expression for ρ^{m+1} , we define the log-likelihood:

$$\mathcal{L}^{m+1}(\beta) := \frac{1}{N} \sum_{i=1}^N \log \mu(x_i^{m+1}) + \log |J^{m+1}(x_i^0)|$$

Transport Map

The particles are updated via the map:

$$T^{m+1}(x^m) = x^m + \beta^{m+1} \cdot f^{(m+1)}(x^0 - x^m)$$

where:

- x^m is the particle position at iteration m ,
- x^0 is the original sample,
- $f^{(m+1)}$ is a flow direction field.

Updating β

To improve the fit between ρ^{m+1} and ρ , we maximize $\mathcal{L}^{m+1}(\beta)$ with respect to β . This yields the update:

$$\beta^{m+1} = \frac{\partial \mathcal{L}^{m+1}}{\partial \beta}$$

This tells us **how fast** $\mathcal{L}^{m+1}$ changes **if we slightly increase or decrease** β .

- If this derivative is **positive**, then increasing β makes $\mathcal{L}^{m+1}$ larger (good).
- If it is **negative**, then decreasing β makes $\mathcal{L}^{m+1}$ larger (also good).
- The size of the derivative tells us **how strongly** β should be adjusted.

So the update:

$$\beta^{m+1} = \frac{\partial \mathcal{L}^{m+1}}{\partial \beta}$$

means: “Update β by the amount that improves the log-likelihood the most **in this step**.”